

Entrega 3: Prioridades, Grafos y Algoritmos

- *Explicar el concepto de cola de prioridades y heap binaria. Justificar su uso en la gestión de misiones o combates.*
- *Describir la representación de grafos y su aplicación en universos de ciencia ficción (planetas, alianzas, rutas espaciales).*
- *Analizar la complejidad de los algoritmos de recorrido (DFS, BFS) y su aplicabilidad en la búsqueda de aliados o enemigos.*

Una **cola de prioridades** es una estructura de datos donde cada elemento tiene asignada una prioridad.

A diferencia de una cola normal (*FIFO "First In, First Out" (Primero en entrar, primero en salir)*), aquí el elemento más importante sale primero, sin importar el orden en el que fue agregado.

La forma más eficiente de implementarla es usando un **Heap Binario**. Este mismo es un árbol binario especial que siempre está "completo", es decir, todos sus niveles del árbol están llenos exceptuando quizá el último. Y un nodo tiene siempre prioridad sobre sus hijos, existiendo dos tipos:

- **Min-Heap:** el valor más pequeño está arriba
- **Max-Heap:** el valor más grande está arriba

Al usar Python, por defecto un Min-Heap, en nuestro código multiplicamos nuestras prioridades por -1 para que el número más grande sea tratado como el más importante. Por ejemplo, en Dragon Ball, las misiones pueden tener distintos niveles de amenaza:

- 10 → *Freezer destruyendo planetas*
- 7 → *Cell absorbiendo ciudades*
- 5 → *Babidi intentando despertar a Majin Buu*

Acá, la cola de prioridades garantiza que la misión de nivel 10 se asigne primero, aunque haya sido la última en agregarse.

Esto es exactamente lo que hace nuestro **Gestor de Misiones**, usando `heapq`.

El **uso de una cola de prioridades en la gestión de misiones/combates**, es ideal cuando queremos atender primero lo más urgente.

Por ejemplo, si Goku recibe tres alertas de ataque, la prioridad va a depender de la amenaza o recompensa en la que finalice ese ataque, no va a depender del orden en el que aparecieron.



Trabajo Final para la Cátedra: Estructuras de Datos en Python

Integrantes: Álvarez Rocío, Varela Ian, Gómez Morena.

Esto también es útil para simular, por ejemplo: alertas de poder que detecta Kaio-sama, orden de turnos en combates, selección del villano que atacará primero, entre otros.

Esta clase de estructura simplifica la toma de decisiones automáticas del sistema.

Representación de Grafos y su aplicación en Universos de Ciencia Ficción

Un **grafo** es una estructura formada por *Nodos* (puntos (planetas, bases, naves, guerreros)) y por *Aristas* (conexiones entre esos puntos (rutas, alianzas, conflictos)).

Los grafos son realmente útiles en la ciencia ficción porque permiten representar, por ejemplo:

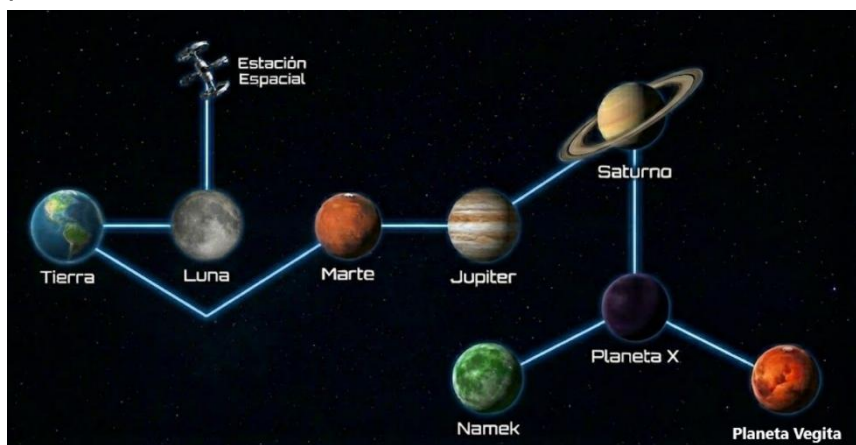
- Sistemas de transporte interplanetario
- Redes de alianzas entre razas
- Mapa del multiverso
- Vínculos entre guerreros (Goku ↔ Vegeta, Gohan ↔ Piccolo)

Recorridos DFS y BFS

DFS (Búsqueda en Profundidad) "Depth-First Search": Explora un camino hasta el fondo, y recién ahí vuelve atrás. Su complejidad está en: $O(V+E)$ $\{V=\text{cantidad de nodos} ; E=\text{cantidad de rutas}\}$, es decir que recorrerá todos los nodos y sus rutas. Las ventajas que tiene son que: es ideal para explorar zonas desconocidas, y es útil para ver si existe alguna conexión entre dos o más lugares.

Por ejemplo: DFS "Depth-First Search" es como cuando Goku debe explorar un planeta nuevo, avanzando siempre hacia lo desconocido hasta que no puede más lejos.

En nuestro trabajo, este recorrido permite recorrer todo el universo conectando planetas.



Trabajo Final para la Cátedra: Estructuras de Datos en Python

Integrantes: Álvarez Rocío, Varela Ian, Gómez Morena.

BFS (Búsqueda en Amplitud) "Breadth-First Search": Explora primero los planetas cercanos (misma "capa"), luego los que están un nivel más lejos. Su complejidad es la misma que en DFS donde $O(V+E)$. Sus ventajas son que encuentra el camino más corto (menos saltos), y que es útil para navegación espacial y mapas.

Por ejemplo, BFS "Breadth-First Search" es como cuando Bulma calcula la ruta más eficiente a Namek.

En la entrega, este recorrido devuelve la ruta más corta entre dos planetas.

El sistema combina estas estructuras para modelar un universo dinámico, donde: las amenazas surgen con diferentes prioridades, los planetas están interconectados, y los guerreros deben elegir caminos óptimos para responder.

